TESTES DE SOFTWARE · MACHINE LEARNING

Fuzz Testing em Machine Learning

ResNet-50, ViT-Base e o tokenizer do DistilBERT testados com fuzzing guiado por cobertura

⚠ **Conteúdo gerado por IA (Claude):** código, testes, textos e slides. Revise antes de usar ou apresentar.

28 de outubro de 2026 · leonardorh18/ml-fuzzy-tests

O QUE É FUZZ TESTING

Bombardear o sistema com entradas até algo quebrar

Em vez de escrever casos de teste à mão, um programa (o **fuzzer**) gera milhares de entradas automaticamente e observa o que acontece.

Exemplo clássico

Um leitor de PDF recebe milhares de arquivos levemente corrompidos. Um deles faz o programa travar: é um bug encontrado antes que um usuário ou um atacante o encontre.

1. Gerar uma entrada
aleatória ou mutando uma que já existe

↓

2. Executar o sistema
e medir o que ele exercitou

↓

3. Verificar: quebrou?
a regra que decide é o **oráculo**

↓

4. Guardar se explorou algo novo
guiado por **cobertura**; repete milhares de vezes

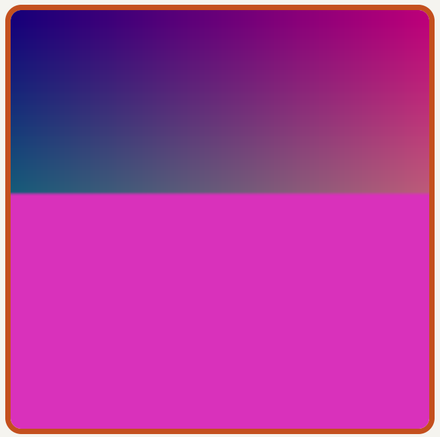
FUZZ TESTING EM MACHINE LEARNING

Em ML, a falha é o modelo mudar de ideia



Original

ViT: classe 614



Mutada (brilho)

ViT: classe 727



Diferença ×6

≈ 0,08 por pixel

O que muda em relação ao fuzzing clássico

Entrada

imagens e textos levemente alterados

Oráculo

o modelo raramente trava; falha é mudar a predição sem motivo

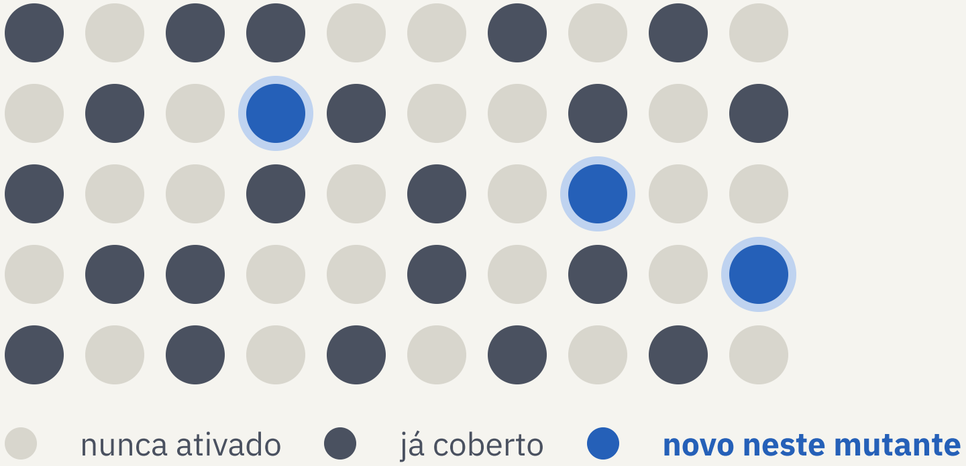
Cobertura

neurônios ativados, não linhas de código

Exemplo real do nosso teste (iteração 20) · classes numeradas porque o ViT rodou com pesos aleatórios · Conteúdo gerado por IA · 3

COMO FUNCIONA · COBERTURA DE NEURÔNIOS

O fuzzer guarda as imagens que acendem neurônios novos



Esquema ilustrativo de uma camada

Neurônio

um canal de um bloco da ResNet ou uma dimensão de um bloco do ViT

Coberto

alguma entrada já o ativou acima de 50% do máximo da camada

Feedback

mutante sem falha que acende neurônios novos volta ao corpus e gera novos mutantes

Ideia do DeepXplore (2017) e do TensorFuzz (2019)

MODELOS USADOS

Dois classificadores de imagem e um tokenizer de texto

ResNet-50

Para que serve: dizer qual objeto aparece numa foto, entre 1000 classes do ImageNet.

Como: rede convolucional de 50 camadas; filtros detectam bordas, texturas e formas.

```
foto de um cachorro
→ "golden retriever"
```

ViT-Base

Para que serve: a mesma tarefa da ResNet, com as mesmas 1000 classes.

Como: corta a foto em quadrados de 16×16 pixels e usa atenção, a técnica dos modelos de linguagem.

```
foto de uma xícara
→ "coffee mug"
```

Tokenizer

Para que serve: quebrar texto em pedaços (tokens) e trocar cada um por um número que o DistilBERT entende.

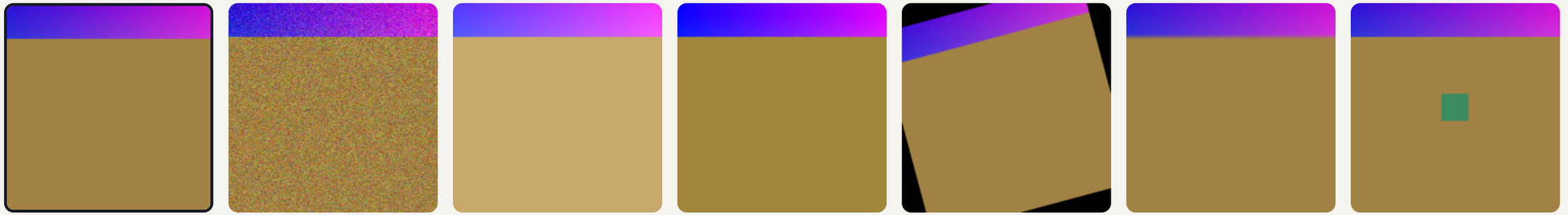
Por que testar: todo texto passa por ele antes do modelo.

```
"the cat is sleeping"
→ [CLS] the cat is sleeping
[SEP]
```

ResNet e ViT fazem a mesma tarefa com arquiteturas diferentes, o que permite comparar as duas com as mesmas imagens.

COMO GERAMOS OS TESTES · IMAGENS

Seis mutações, cortadas para no máximo 0,08 por pixel



original ruído brilho contraste rotação blur patch

1. Sorteia uma imagem do corpus

2. Aplica uma mutação aleatória

3. Corta cada pixel a ±0,08 da original

4. Classifica e mede neurônios

5. Classe mudou? **Falha**, salva a imagem

Mutações no máximo da faixa, antes do corte em 0,08 · semente sintética usada no teste · Conteúdo gerado por IA · 6

COMO GERAMOS OS TESTES · TEXTO

O Atheris gera bytes; quatro oráculos julgam o tokenizer

bytes gerados
`0a 91 01 00 00 00`

→

viram entrada
`max_length=2, 'Ƒ'`

→

tokenizer devolve
`[CLS] x [SEP]`

1. Sem exceção
a chamada nunca pode quebrar

2. Respeita o limite
tokens ≤ max_length; aqui, 3 > 2

3. Ids válidos
todo número existe no vocabulário

4. Ida e volta estável
decodificar e codificar de novo não muda mais nada

O Atheris guarda as entradas que executam linhas novas do código Python do tokenizer e muta a partir delas: é o fuzzing clássico, guiado por cobertura de código.

CONFIGURAÇÃO DO EXPERIMENTO

Mesmas condições para os dois modelos

Parâmetro	Valor
Iterações	500 por modelo
Sementes	6 imagens sintéticas
Perturbação máxima (ε)	0,08 por pixel
Limiar de cobertura	0,5
Tempo em CPU	17 s ResNet · 43 s ViT
Tokenizer	61 s rápido · < 1 s Python

Pesos aleatórios

O ambiente não acessava o Hugging Face. Os modelos rodaram com a arquitetura real e **pesos aleatórios**, e o tokenizer com um vocabulário local.

O teste prova que o método funciona; os números não medem a robustez dos modelos treinados.

RESULTADOS · IMAGENS

ViT: 45 falhas em 500. ResNet: nenhuma.

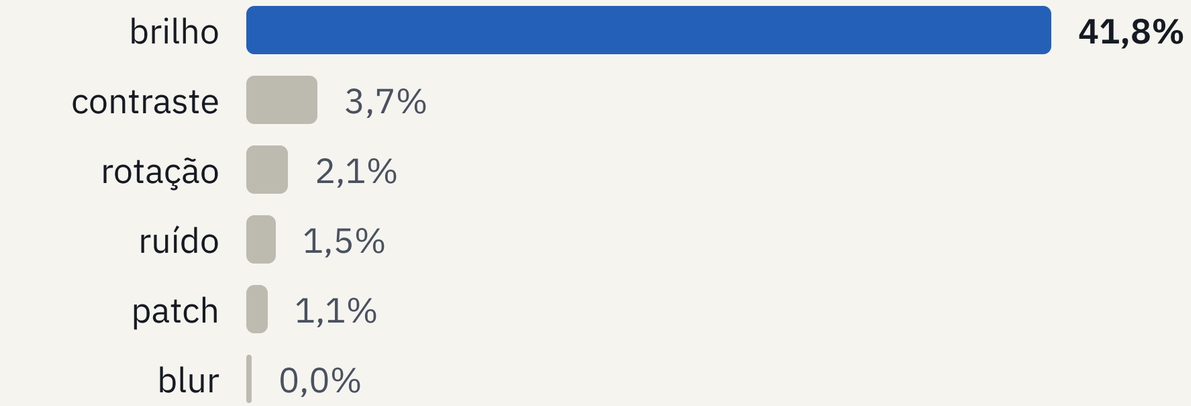
ViT-Base

45 / 500

ResNet-50

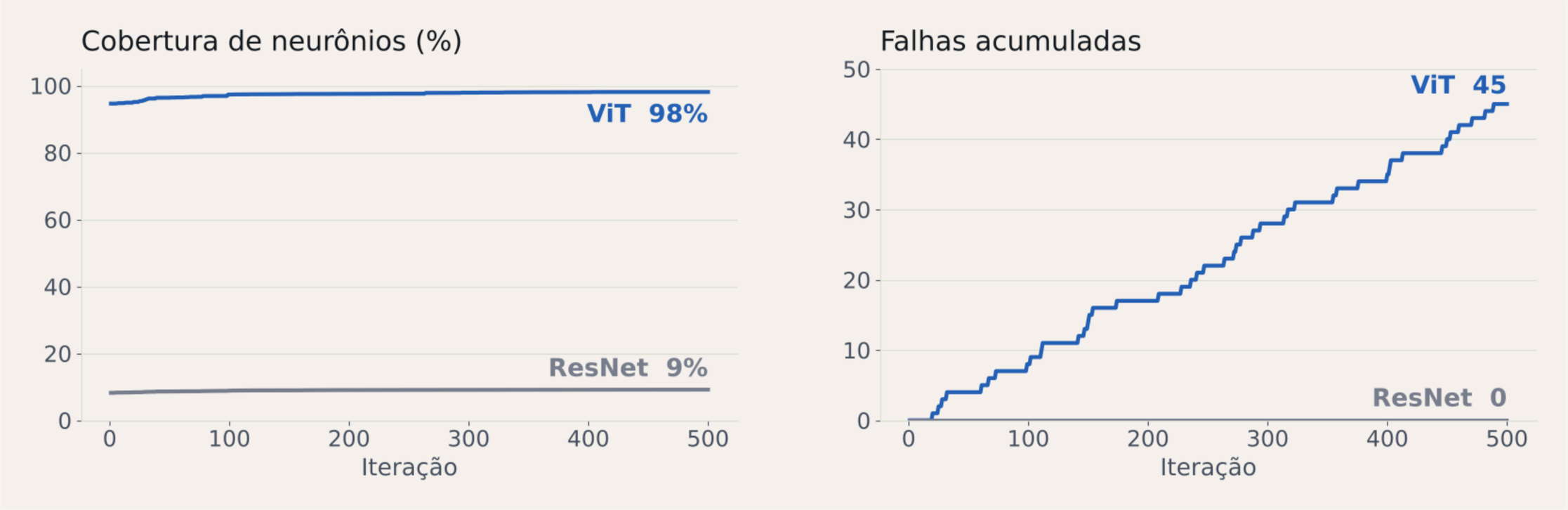
0 / 500

No ViT, o brilho causou 38 das 45 falhas



RESULTADOS · COBERTURA × FALHAS

Mais cobertura não trouxe mais falhas



A cobertura do ViT já começou em 94,6% e a da ResNet travou em 9%. As falhas do ViT cresceram em ritmo constante, sem acompanhar a cobertura (correlação de −0,31).

RESULTADOS · TOKENIZER

O tokenizer em Python estourou o max_length

Python puro · max_length=2 · texto 'Ƒ'

`[CLS] x [SEP]`

3 tokens: falha, o truncamento desiste

Rápido (Rust) · mesma entrada

`[CLS] [SEP]`

2 tokens: correto

Execução	Execuções	Cobertura	Falhas
Rápido (Rust)	258.995	141	0
Python puro	~470	295	**1, em < 1 s**

Cobertura = trechos de código Python vistos pelo Atheris · vocabulário local · Conteúdo gerado por IA · 11

CONCLUSÃO

Respostas às perguntas do trabalho

Pergunta	Resposta
Mudanças pequenas trocam a classe?	Sim no ViT: 9% dos mutantes, com ε = 0,08
Qual mutação quebra mais?	Brilho: 42% de falha no ViT
ResNet ou ViT é mais robusto?	Inconclusivo: pesos aleatórios
Mais cobertura, mais falhas?	Não: correlação de −0,31
O tokenizer aguenta qualquer texto?	O rápido sim; o em Python estoura o max_length

Próximo passo: rodar com os pesos reais do Hugging Face e fotos de verdade.